Projeto do Robô: Entrega 1

Nov 5, 2026 · @Jemerson Damásio

Visão geral

Seu grupo vai escrever um programa que encontra a rota mais barata de um robô em uma grade, usando recursão, e explicar por que ele funciona. Memoização, tabela e a lista de coordenadas da rota ficam para a Entrega 2.

Item	Definição
Valor	3,0 pontos
Grupo	6
Linguagem	À escolha do grupo: Python, JavaScript, Java ou outra com recursão

O problema

O robô sai do canto superior esquerdo, chega ao canto inferior direito e deve gastar o mínimo de energia.

	Coluna 0	Coluna 1	Coluna 2	Coluna 3	Coluna 4
Linha 0	**2** (partida)	1	9	9	9
Linha 1	3	X	1	1	9
Linha 2	1	1	1	X	2
Linha 3	9	9	1	1	**1** (chegada)

- O robô anda uma casa por vez, só para a **direita** ou para **baixo**.
- Ele não sai da grade e não entra em obstáculos, marcados com `X`.
- O custo é a soma de todas as casas visitadas, **incluindo a partida e a chegada**.

Nesta grade, a melhor rota desce pela primeira coluna, segue pelas casas de custo 1 e custa 2 + 3 + 1 + 1 + 1 + 1 + 1 + 1 = **11**. Em algumas grades os obstáculos bloqueiam tudo, e o programa deve informar que não há caminho.

O contrato

Toda linguagem usa os mesmos formatos, aqui apresentados em JSON. A entrada é um texto com a chave `custos`, e `null ou None` marca o obstáculo:

```
{"custos": [[2, 1, 9, 9, 9],
            [3, null, 1, 1, 9],
            [1, 1, 1, null, 2],
            [9, 9, 1, 1, 1]]}
```

A entrada é válida quando `custos` é uma lista não vazia de linhas de mesmo tamanho, não vazias, e cada valor é um inteiro positivo ou `null`. Usem o leitor de JSON da linguagem ou uma biblioteca. A recursão e as regras do problema precisam ser escritas pelo grupo.

A função `resolver(entrada)` devolve um resultado com estes campos, no tipo adequado da linguagem:

Situação	Saída
Existe rota	`{"status": "ok", "custo": 11}`
Entrada válida, sem rota	`{"status": "sem_caminho", "custo": null}`
Entrada inválida	`{"status": "entrada_invalida", "erro": "mensagem"}`

A Aula 4 também mostra o campo `caminho`, mas ele só será exigido na Entrega 2. Não o incluam agora.

O que fazer

São oito tarefas, nesta ordem.

1. Descrever o problema. Com as palavras do grupo, expliquem entrada, saída, objetivo e regras. Deem um exemplo próprio de entrada válida sem rota e um de entrada inválida.

2. Resolver o caminhamento com recursão. A função `recursivo(mapa)` devolve o menor custo ou indica que não há rota. Obstáculos e posições fora da grade contam como impossíveis. Na chegada, a resposta é o custo da própria casa. Não guardem resultados de chamadas anteriores.

3. Testar. Escrevam testes que rodem sozinhos, com todos os casos abaixo e pelo menos três criados pelo grupo.

Entrada	Resultado esperado
`custos` = `[[7]]`	`ok`, custo 7
`custos` = `[[1, 2, 3]]` ou `[[1], [2], [3]]`	`ok`, custo 6
`custos` = `[[1, 1, 1], [1, 1, 1]]`	`ok`, custo 4
Grade do exemplo	`ok`, custo 11
`custos` = `[[1, 1, null], [2, null, 1], [1, 1, 1]]`	`ok`, custo 6
`custos` = `[[null]]`, `[[null, 1], [1, 1]]`, `[[1, 1], [1, null]]` ou `[[1, null], [null, 1]]`	`sem_caminho`
`custos` = `[]`, `[[]]` ou `[[1, 2], [3]]`	`entrada_invalida`
`custos` = `[[true, 1]]`, `[["1", 1]]`, `[[1.5, 1]]`, `[[0, 1]]` ou `[[-1, 1]]`	`entrada_invalida`
Texto `{"matriz": [[1]]}`, `[[1, 2]]` ou `{"custos": [[1, 2]`	`entrada_invalida`

4. Explicar por que funciona. Respondam em poucas frases: por que a recursão sempre termina? Por que ela encontra o menor custo? O que acontece quando não há rota? Dica: a cada chamada falta um movimento a menos, e toda rota começa indo para a direita ou para baixo.

5. Medir o trabalho. Contem as chamadas da função nas grades abaixo, todas com casas de custo 1, e também na grade do exemplo. Se o programa só chama vizinhos dentro da grade, os números devem bater. Se não baterem, expliquem a diferença.

Grade	2 × 3	3 × 3	6 × 6	10 × 10	1 × 10
Chamadas	9	19	923	184.755	10

6. Implementar a regra gulosa. Implemente uma função `guloso(mapa)`, que vai sempre para o vizinho livre mais barato e prefere a direita no empate. Confirmem que ela dá 26 na grade do exemplo, contra 11. Depois, criem uma grade própria de até 4 × 4 em que ela também erre e expliquem onde a escolha atrapalhou.

Entregáveis

Cada grupo entrega três itens.

Entregável	Formato	Conteúdo
Código	`entrega1_grupoNN.zip`, repositório ou notebook	Funções `recursivo`, `resolver` e `guloso`, e testes na linguagem escolhida.
Relatório	`entrega1_grupoNN.pdf`	Nome dos Integrantes, e resultados conforme descrição acima

Como será avaliado

A nota soma cinco critérios, com crédito parcial em cada um. A apresentação não tem nota separada e serve de evidência para os critérios.

Critério	Pontos	O que se espera
Descrição do problema	0,5	Entrada, saída e regras sem ambiguidade, com os dois exemplos próprios
Recursão e testes	1,0	Resposta correta, sem guardar resultados, e testes que passam
Por que funciona	0,6	Explicação de por que termina, acerta e detecta a falta de rota
Medição do trabalho	0,6	Contagem e explicações coerentes com o código entregue
Regra gulosa	0,3	Grade própria, custos e explicação de onde a regra errou